

Real-time systems

Real-time systems are computer systems designed to respond to input and produce output within a specified timeframe, known as a deadline. The primary characteristic that distinguishes real-time systems from other computer systems is their ability to guarantee timely and predictable responses to events or tasks. These systems are widely used in various domains, including industrial control, automotive systems, aerospace, medical devices, multimedia, and telecommunications.

There are two main types of real-time systems.

Hard real-time systems: Meeting deadlines is crucial in hard real-time systems. Failure to complete a task within its specified deadline can lead to dangerous consequences, system failure, or compromise safety. Examples include flight control systems in aeroplanes, medical devices for life support, and airbag deployment in vehicles.

Soft real-time systems: In soft real-time systems, meeting deadlines is important, but occasional misses may be tolerable without dangerous consequences. While timely execution is desirable, the system can still provide useful results even if some deadlines are missed. Examples of soft real-time systems include streaming multimedia applications, online video conferencing, and online gaming. The user experience may degrade if deadlines are missed, but it doesn't lead to system failure or severe safety issues.

Apart from these main types, there is a middle ground known as 'firm real-time systems', a hybrid between hard and soft real-time systems. Firm real-time systems prioritise meeting most deadlines, but occasional misses are allowed if they don't significantly impact the overall system behaviour. These systems are often used in multimedia streaming and telecommunications applications.

QNX

QNX is a real-time operating system known for its strong real-time characteristics, reliability, and low and deterministic latencies. Originally developed by QNX Software Systems, it is now a part of BlackBerry Limited and is widely used across various industries and applications that require real-time performance, determinism, and high reliability. Its characteristics are:

Deterministic behaviour: QNX is designed to be highly deterministic, guaranteeing that critical tasks and processes meet their deadlines consistently. This attribute is particularly crucial in safety-critical applications, such as automotive systems, industrial automation, and medical devices.

Real-time scheduling: QNX employs a priority based preemptive scheduler, allowing tasks with higher priorities to preempt lower-priority tasks. This scheduling mechanism ensures that time-critical tasks receive immediate attention without interference from lower-priority tasks.

Fault tolerance and reliability: QNX is designed with a strong focus on fault tolerance and reliability. Its microkernel architecture isolates critical components, reducing the impact of failures and enhancing system resilience.

Resource management: QNX provides efficient resource management, allowing developers to allocate CPU time, memory, and other system resources to tasks in a controlled and predictable manner. This capability is essential for meeting the timing requirements of real-time applications.

Preemption: QNX supports preemption, enabling a higher-priority task to interrupt the execution of a lower-priority task promptly.

Fast interrupt handling: QNX provides efficient interrupt handling mechanisms. When an interrupt occurs, the kernel quickly determines the appropriate handler for the interrupt and dispatches it. This minimises the time spent in the interrupt handling routine.

Latencies: QNX is known for its low and deterministic latencies, making it suitable for real-time applications with strict timing requirements. The latencies in a real-time operating system like QNX refer to the time it takes for the system to respond to an event or an interrupt and start executing a specific task. There are several factors that can influence latencies in QNX:

- Context switching
- Task scheduling
- Kernel design
- Interrupt handling

It's important to note that while QNX offers low and deterministic latencies, the actual latencies experienced in a specific system can depend on various factors, such as the hardware platform, system workload, and application design. The above techniques are used to optimise systems for low latencies and predictable behaviour.

Linux

Linux also possesses some real-time capabilities. Examples include EDF and PREEMPT_RT patch.

EDF: EDF (earliest deadline first) is a scheduling algorithm commonly used in real-time operating systems and some real-time extensions for general-purpose operating systems, including Linux. It prioritises tasks based on their deadlines, giving preference to tasks with the earliest deadlines for execution.

In standard Linux, the default scheduling algorithm is the completely fair scheduler (CFS), which is designed for fairness and responsiveness in non-real-time workloads. However, there are ways to implement EDF in Linux for real-time applications. One way to enable EDF scheduling in Linux is by using the PREEMPT_RT (real-time) patch.

PREEMPT_RT patch: The PREEMPT_RT patch is a set of patches for the Linux kernel that aim to turn Linux into a soft real-time operating system. The goal of the PREEMPT_RT patch is to improve the responsiveness and predictability of the